The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314
Lab Assignment 1: Solution

1. Project Identification

Item	Details
Project Name	LNMIIT Degree Audit System
Project Type	Web-based system that manages academic requirements for graduation
Team ID	T030
Team Members	Hatwar Manas Vinod, Dhruv Gupta, Aradhya Raina, Kota Sai Vignesh
Roll Numbers	24UCC175, 24UCC176, 24UCC177, 24UCC178
Development Model	Agile + Incremental Development
Primary Users	Students, Registrar, Academic Administrators, System Administrators
Development Team	Student software development team
Course	Software Engineering
Project Objective	To develop a platform that keeps records of students' academic progress and automatically evaluates its completion status and requirements to be eligible for graduation degree attainment
Expected Outcome	A properly working, tested and documented software system developed according to requirements

2. Project Background

Current Problem

Academic progress tracking at LNMIIT is currently managed through disconnected mechanisms such as manual transcript reviews, static PDF catalogs, spreadsheets and paper-based records. This can make it difficult for students to understand their degree completion status, plan future courses, explore program changes, and prepare for graduation. Academic advisors lack real-time visibility into student progress, and the Registrar office must manually verify graduation requirements for each student during peak graduation periods.

Proposed Solution

The LNMIIT Degree Audit System will provide a centralized platform for managing the complete academic progress lifecycle—from automated transcript analysis and requirement validation to what-if scenario exploration, course planning, and official graduation clearance reporting.

3. Project Vision

The LNMIIT Degree Audit System aims to provide a centralized, user-friendly and reliable platform for tracking academic progress, validating degree requirements, exploring program changes, and planning course sequences to ensure timely graduation.

4. Problem Statement

LNMIIT conducts undergraduate and graduate academic programs across four departments involving diverse degree requirements, major/minor combinations, transfer credits, and catalog year variations. Existing progress tracking practices rely on multiple disconnected tools and manual processes, resulting in difficulties in requirement validation, course planning, what-if analysis, graduation clearance, academic advising efficiency, and early identification of at-risk students. The LNMIIT Degree Audit System is proposed as an integrated software solution to streamline these activities through a single automated platform.

5. Project Objectives

The initial objectives should be:

1. provide a centralized platform for tracking students' academic progress
2. Provide role-based access control
3. Allow authorized administrators to define, update, and maintain degree requirements, course categories, and credit thresholds.
4. Automate degree requirement comparison
5. Automate credit and deficiency analysis b
6. Provide students with secure access to their real-time degree requirement completion status
7. Generate degree audit reports s
8. Maintain data integrity and security
9. To support all LNMIIT departments and multiple catalog years.
10. To develop the system incrementally using software engineering practices.

6. Stakeholders

Stakeholder	Interest / Role
Students (UG & PG)	View degree progress, plan courses, explore major/minor options, check graduation eligibility
Faculty Advisors	Monitor advisee academic progress, provide course planning guidance, approve semester registrations
HODs (CCE, CSE, ECE, MME Departments)	Review departmental graduation requirements, verify BTP/thesis completion, approve degree conferral
Registrar Office	Final graduation clearance authority, maintain official catalog requirements, issue degree certificates
Admission Cell	Evaluate and record transfer credits for lateral entry, approve course substitutions and equivalencies
Dean of Academic Affairs	Oversee institutional academic standards, review policy compliance, handle exception requests
System Administrator	Manage user accounts and roles, configure system settings, monitor technical performance and security
System Development Team	Design, develop, test and maintain the LNMIIT Degree Audit System

7. Initial Scope

The Degree Audit System will initially cover:

- User authentication/login
- Role-based access
- User account and role management by system administrators
- Semester grades and course information imported by administrators
- Automatic comparison of completed courses/credits against curriculum requirements
- Progress dashboard
- Requirement breakdown for different degrees
- Multi-catalog year support
- Prerequisite verification
- Audit report generation (view/download)
- Advisor monitoring dashboard
- At-risk student alerts
- Basic analytics/reports

Initially Out of Scope

To keep the project manageable, we should initially exclude things such as:

- What-if degree planning simulators or AI-based course recommendations
- Mobile native application (web-based only initially)
- Support for multiple universities or cross-institution transfer credit evaluation
- Automated official transcript generation for external use
- Complete ERP functionality
- Real-time faculty/grade-entry integration (grades assumed to be entered via existing academic records before import)

8. Initial Assumptions

- Academic catalogs accurately reflect official degree requirements.
- Student transcript data is available for import.
- Program requirements remain relatively stable within an academic year
- The system will initially be developed as a web application.
- The system will use a centralized database.
- LNMIIT IT will provide hosting infrastructure and technical support.
- System-generated audit reports will be accepted as official by Registrar.
- Students may choose catalog from admission year forward.

9. Initial Constraints

- Limited development time within the semester.
- Limited access to real institutional records
- Degree requirements may differ across academic programs.
- The system will initially be developed and tested using a limited set of degree programs and sample
- Requirement changes may occur during the development process.
- Limited testing with real students, faculty, and academic administrators.

- Student academic records and degree-related information need to be appropriately protected.
- Requirement changes may occur during the development process.
- Integration with existing institutional academic/ERP systems may not be available during development.
- Limited infrastructure and resources available for development and testing.
- The project is developed by a student team, limiting the time and resources available for implementing a large-scale institutional system.

10. Initial Success Criteria

- different user roles can access appropriate functionality.
- student transcripts are accurately imported and validated.
- students can view progress dashboards showing completed and remaining requirements.
- what-if analysis allows exploring major/minor changes.
- The system supports all departments and multiple catalog years.
- transfer credits and course substitutions are tracked correctly.
- official audit reports are generated and accepted by Registrar office.
- advisors can monitor advisees and identify at-risk students.
- important system functions are tested.
- defects are documented and resolved.
- The project is maintained in a controlled Git/GitHub repository.
- The final system is accompanied by appropriate software engineering documentation.

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314
Lab Assignment 2: Solution

11. User Classes

11.1 Student

Students are the primary users who track their own academic progress.
They shall be able to:

- authenticate to the system.
- view their degree audit report.
- view completed courses and credits.
- view in-progress courses.
- view remaining requirements.
- view progress by requirement categories (core, PE, DE, OE, HSS, major, minor)
- perform what-if analysis for major/minor changes.
- view prerequisite chains.
- view transfer credit evaluations.
- download/print audit reports.
- view graduation eligibility status.

11.2 Faculty Advisor

Faculty advisors guide students through their academic journey.
The advisor shall be able to:

- view audit reports for their assigned students.
- monitor student progress across all requirement categories.
- identify at-risk students based on progress.
- view student course history;
- access multi-student progress summaries.
- approve advised course selections.
- view what-if scenarios for advisees.
- generate advisor reports.

11.3 Head of Department (HOD)

HODs oversee departmental academic progress and graduation readiness.
The HOD shall be able to:

- view department-level progress analytics.
- monitor graduation-eligible students.
- review BTP/thesis completion status.
- Access progress reports.
- verify departmental requirement compliance.
- view at-risk student lists.

- generate department reports.

11.4 Registrar Office

The Registrar office is the final authority for graduation clearance.
The Registrar shall be able to:

- manage catalog requirements for all programs.
- import and update student transcripts.
- view graduation eligibility reports.
- generate official audit reports.
- manage catalog year assignments.
- approve course substitutions.
- verify graduation requirements.
- generate institution-wide reports.

11.5 Admission Cell

The Admission Cell handles transfer credits and lateral entries.
They shall be able to:

- record transfer credits from other institutions.
- map transfer courses to LNMIIT requirements.
- approve course substitutions.
- manage credit-by-examination.
- view transfer credit impact on audit.
- generate transfer credit reports.

11.6 Dean of Academic Affairs

The Dean oversees academic standards and handles exceptions.
The Dean shall be able to:

- review policy compliance reports.
- approve exception requests.
- access institution-wide analytics.
- monitor academic standards.
- view trend analysis across programs.

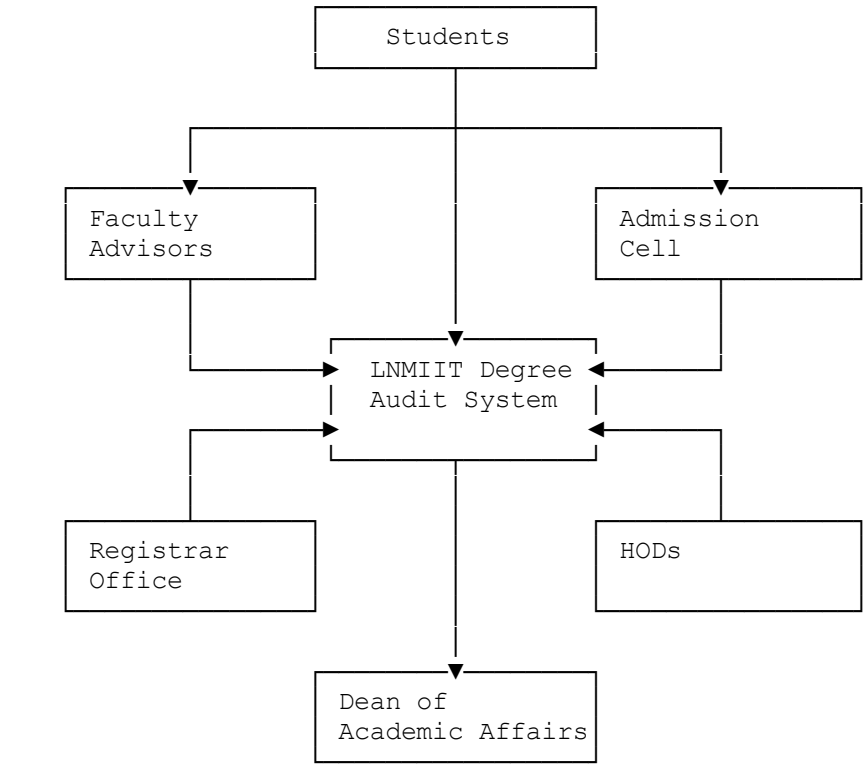
11.7 System Administrator

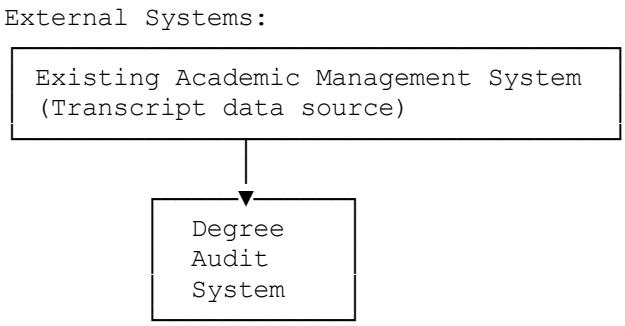
System administrators manage the technical system.
The administrator shall be able to:

- manage user accounts.
- assign user roles.
- configure system settings.
- manage catalog data.
- monitor system activity.
- access system logs
- generate adminisetrative reports.

12. System Context

The System Context Diagram shows the LNMIIT Degree Audit System and its interactions with external actors and systems.
[System Context Diagram]





External Actors:

- Students, Faculty Advisors, HODs, Registrar Office, Admission Cell, Dean
- Existing Academic Management System (for transcript data import)

Note: Diagram saved as `docs/requirements/diagrams/System-Context.png`

13. Functional Requirements

13.1 Authentication and Access Management

FR-001 — User Login
The system shall allow a registered user to authenticate using valid LNMIIT credentials.

FR-002 — Authentication Failure
The system shall notify the user when authentication fails without revealing sensitive authentication information.

FR-003 — Logout
The system shall allow authenticated users to terminate their session.

FR-004 — Role Assignment
The system shall allow authorized administrators to assign roles to users.

FR-005 — Role-Based Access
The system shall restrict system functions according to the user's assigned role.

13.2 Catalog and Requirement Management

FR-006 — Define Degree Requirements
The system shall allow authorized Registrar staff to define degree requirements for each program.

FR-007 — Requirement Categories
The system shall allow specification of requirement categories: core courses, program electives (PE), departmental electives (DE), open electives (OE), HSS courses, major requirements, minor requirements, and certificate requirements.

FR-008 — Credit Thresholds
The system shall allow specification of minimum credit requirements for each category.

FR-009 — GPA Requirements
The system shall allow specification of minimum GPA requirements for degree completion.

FR-010 — Catalog Year Management
The system shall support multiple catalog years, allowing students to be audited against their applicable catalog.

FR-011 — Prerequisite Definition
The system shall allow definition of prerequisite relationships between courses.

FR-012 — Course Equivalency
The system shall allow definition of course equivalencies and substitutions.

13.3 Transcript Management

FR-013 — Import Transcripts
The system shall allow authorized Registrar staff to import student transcript data.

FR-014 — Update Transcripts
The system shall allow authorized staff to update student transcripts with new course completions.

FR-015 — Course History
The system shall maintain a complete history of courses taken, including grades and credits earned.

FR-016 — In-Progress Courses
The system shall track courses currently in progress.

FR-017 — Grade Validation
The system shall validate imported grades against acceptable grade values.

13.4 Degree Audit Processing

FR-018 — Automated Audit Calculation
The system shall automatically calculate degree audit status when transcript data changes.

FR-019 — Requirement Matching
The system shall match completed courses against degree requirements.

FR-020 — Credit Calculation
The system shall calculate total credits earned and remaining in each category.

FR-021 — GPA Calculation
The system shall calculate cumulative and major GPA.

FR-022 — Deficiency Identification
The system shall identify unmet requirements and deficiencies.

FR-023 — Graduation Eligibility
The system shall determine graduation eligibility based on requirement completion.

FR-024 — Prerequisite Verification
The system shall verify prerequisite satisfaction for courses.

13.5 Progress Dashboard

FR-025 — Student Dashboard
The system shall provide students with a visual progress dashboard.

FR-026 — Completion Status
The system shall display completion status for each requirement category (satisfied, in-progress, not-started).

FR-027 — Progress Percentage
The system shall display overall degree completion percentage.

FR-028 — Requirement Breakdown
The system shall display detailed breakdown of requirements by category.

FR-029 — Course List Display
The system shall display lists of completed, in-progress, and required courses.

FR-030 — Visual Indicators
The system shall use clear visual indicators (colors, icons) to show requirement status.

13.6 Transfer Credit Management

FR-031 — Record Transfer Credits
The system shall allow Admission Cell to record transfer credits from other institutions.

FR-032— Map Transfer Courses
The system shall allow mapping of transfer courses to LNMIIT requirements.

FR-033 — Transfer Credit Approval
The system shall track approval status of transfer credit evaluations.

FR-034 — Course Substitution
The system shall allow recording of approved course substitutions.

FR-035 — Transfer Credit Display
The system shall display transfer credits separately in audit reports.

13.7 Advisor Functions

FR-036 — Advisee List
The system shall display list of assigned students to faculty advisors.

FR-037 — Advisee Audit View
The system shall allow advisors to view audit reports for their advisees.

FR-038 — At-Risk Identification
The system shall identify and flag at-risk students based on progress criteria.

FR-039 — Multi-Student Reports
The system shall allow advisors to generate reports for all advisees.

FR-040 — Progress Alerts
The system shall provide alerts for advisees falling behind requirements.

13.8 Reporting and Analytics

FR-041 — Student Audit Report
The system shall generate detailed audit reports for individual students.

FR-042 — Deficiency Report
The system shall generate reports listing unmet requirements.

FR-043 — Graduation Eligibility Report
The system shall generate reports of graduation-eligible students.

FR-044 — Department Analytics
The system shall generate department-level progress analytics for HODs.

FR-045 — Cohort Reports
The system shall generate cohort-wise progress reports.

FR-046 — Report Export
The system shall allow export of reports in PDF format.

FR-047 — Report Download
The system shall allow users to download their audit reports.

13.9 Administrative Functions

FR-048 — User Management
The system shall allow administrators to create and manage user accounts.

FR-049 — Bulk Import
The system shall allow bulk import of student and course data by elibigible administrators.

FR-050 — System Configuration
The system shall allow configuration of system-wide settings.

FR-051 — Audit Logging
The system shall log all significant system actions.

FR-052 — Exception Handling
The system shall allow Dean to approve exceptions to standard requirements.

14. Business Rules

ID	Business Rule
BR-001	Only authorized Registrar staff can modify catalog requirements
BR-002	Only authorized Registrar staff can import or modify student transcripts
BR-003	Students can only view their own audit reports
BR-004	Advisors can only view audits for their assigned advisees
BR-005	A course can satisfy only one requirement category unless explicitly allowed as double-counted
BR-006	Grades below D cannot count toward degree requirements
BR-007	Transfer credits must be approved before appearing in official audit
BR-008	Course substitutions require appropriate authority approval
BR-009	Catalog year cannot be earlier than student admission year
BR-010	Catalog year cannot be older than 8 years
BR-011	Minimum 120 credits required for BTech degree
BR-012	Minimum CGPA of 5.0 required for graduation
BR-013	Prerequisites must be satisfied before a course counts toward requirements
BR-014	Only Dean can approve exceptions to standard requirements

15. Non-Functional Requirements

15.1 Performance

NFR-001: The system should provide acceptable response times for normal user operations.
NFR-002: Audit calculation should complete within 10 seconds for a typical student transcript.
NFR-003: The system should support at least 500 concurrent users.
NFR-004: Dashboard page load should complete within 5 seconds.

15.2 Security

NFR-005: All functionality shall require authentication.
NFR-006: Access to functionality shall be controlled by user roles.
NFR-007: Student academic data shall not be exposed to unauthorized users.
NFR-008: Administrative operations should be auditable.
NFR-009: Passwords shall be stored using industry-standard hashing.
NFR-010: The system shall protect against common web vulnerabilities.

15.3 Usability
NFR-011: The interface shall use consistent navigation.
NFR-012: Error messages shall be understandable to users.
NFR-013: The dashboard should present information clearly with visual indicators.
NFR-014: Common activities such as viewing audit reports should require minimal interaction.
NFR-015: The system should be accessible to users with disabilitie.

15.4 Reliability
NFR-016: Transcript records shall be maintained reliably.
NFR-017: Audit calculations shall be deterministic and repeatable.
NFR-018: The system shall handle errors gracefully without data loss.

15.5 Availability
NFR-019: The system should be available during normal campus operating periods, except during planned maintenance.
NFR-020: System downtime should be scheduled during low-usage periods.

15.6 Maintainability
NFR-021: The software shall follow a modular architecture.
NFR-022: Source code shall be maintained using version control.
NFR-023: Software documentation shall be maintained alongside relevant system changes.
NFR-024: Code shall follow consistent coding standards.

15.7 Scalability
NFR-025: The architecture should allow the system to support increasing numbers of users and programs.
NFR-026: The database design should accommodate growth in student and course data.

15.8 Compatibility
NFR-027: The system should operate correctly on commonly used modern web browsers (Chrome, Firefox, Safari, Edge).
NFR-028: The system should be responsive and usable on mobile phones and tablets.

15.9 Privacy
NFR-029: Personal academic information shall be accessible only to authorized users.
NFR-030: The system shall comply with applicable data privacy regulations.

15.10 Auditability
NFR-031: Significant administrative actions should be recorded with user ID and timestamp.
NFR-032: Audit logs shall be retained for at least one academic year.

16. Data Requirements
The initial logical information requirements include the following entities.

User
Possible attributes:
- User ID
- Name
- Email
- Role (Student, Advisor, HOD, Registrar, Admin, Admission Cell, Dean)
- Department
- Account status

Student
Possible attributes:

- Student ID (Roll Number)
- User ID (reference)
- Program (BTech, MTech, etc.)
- Department (CCE, CSE, ECE, MME)
- Batch/Year
- Admission Date
- Catalog Year
- Major
- Minor (if applicable)
- Advisor ID
- Current Semester
- Status (Active, Graduated, etc.)

Catalog
Possible attributes:

- Catalog ID
- Catalog Year
- Program
- Department
- Effective Date
- Expiry Date

Requirement
Possible attributes:

- Requirement ID
- Catalog ID
- Requirement Category (Core, PE, DE, OE, HSS, Major, Minor)
- Description
- Minimum Credits
- Minimum GPA
- Course List (if applicable)
- Rules/Conditions

Course
Possible attributes:

- Course Code
- Course Name
- Department
- Credits
- Course Type
- Description
- Prerequisites

Transcript
Possible attributes:

- Transcript ID
- Student ID

- Course Code
- Semester
- Grade
- Credits Earned
- Status (Completed, In-Progress)
- Import Date

TransferCredit
Possible attributes:
- Transfer Credit ID
- Student ID
- Institution Name
- Course Name
- Credits
- Mapped Course Code
- Approval Status
- Approved By
- Approval Date

CourseSubstitution
Possible attributes:
- Substitution ID
- Student ID
- Original Course Code
- Substitute Course Code
- Reason
- Approved By
- Approval Date

AuditReport
Possible attributes:
- Audit ID
- Student ID
- Generated Date
- Catalog Year
- Graduation Eligible (Yes/No)
- Total Credits Earned
- Total Credits Required
- CGPA
- Major GPA
- Status Summary

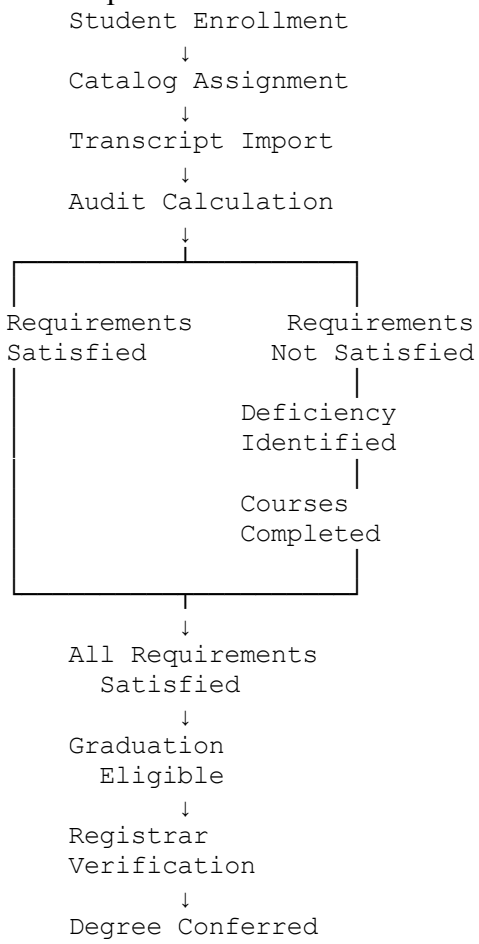
AdvisorAssignment
Possible attributes:
- Assignment ID
- Advisor ID
- Student ID
- Assignment Date
- Status (Active, Inactive)

SystemLog
Possible attributes:

- Log ID
- User ID
- Action
- Timestamp
- Details

17. Degree Audit Lifecycle

The degree audit process follows these states:



State Descriptions:

1. **Student Enrollment:** Student admitted to program
2. **Catalog Assignment:** Student assigned to applicable catalog year
3. **Transcript Import:** Initial transcript data imported
4. **Audit Calculation:** System calculates requirement satisfaction
5. **Requirements Satisfied/Not Satisfied:** Branching based on completion
6. **Deficiency Identified:** Unmet requirements flagged
7. **Courses Completed:** Student completes additional courses
8. **All Requirements Satisfied:** All degree requirements met
9. **Graduation Eligible:** Student eligible for graduation
10. **Registrar Verification:** Final verification by Registrar
11. **Degree Conferred:** Graduation approved

18. Major Use Cases

The initial use-case catalogue is as follows:

ID	Use Case	Primary Actor
UC-001	Login	All users
UC-002	View Degree Audit	Student
UC-003	Download Audit Report	Student
UC-004	View Advisee Progress	Faculty Advisor
UC-005	Identify At-Risk Students	Faculty Advisor
UC-006	Import Student Transcripts	Registrar Office
UC-007	Define Catalog Requirements	Registrar Office
UC-008	Generate Graduation Eligibility Report	Registrar Office
UC-009	Record Transfer Credits	Admission Cell
UC-010	Approve Course Substitution	Admission Cell
UC-011	View Department Analytics	HOD
UC-012	Approve Exception Request	Dean
UC-013	Manage User Accounts	System Administrator
UC-014	View System Logs	System Administrator
UC-015	Update Catalog Requirements	Registrar Office
UC-016	Generate Advisor Reports	Faculty Advisor
UC-017	Map Transfer Courses	Admission Cell
UC-018	View Advisee Audit Report	Faculty Advisor
UC-019	Configure System settings	System Administrator

19. Detailed Use Cases

UC-002 — View Degree Audit
Primary Actor: Student
Preconditions:

- Student is authenticated.
- Student has an assigned catalog year.
- Transcript data exists for the student.

Main Flow:
1. Student selects "View My Audit" from dashboard.
2. System retrieves student's transcript data.
3. System retrieves applicable catalog requirements.
4. System calculates requirement satisfaction.
5. System displays audit dashboard with:
    - Overall progress percentage
    - Requirements by category (satisfied/in-progress/not-started)
    - Completed courses list
    - In-progress courses list
    - Remaining requirements list
    - Graduation eligibility status
6. Student reviews audit information.

Alternative Flows:

- If transcript data is not available → system displays message "No transcript data available."

- If catalog not assigned → system displays message "Catalog year not assigned. Contact Registrar."
- If audit calculation fails → system displays cached audit from last successful calculation.

Postcondition:
Student has viewed their current degree audit status.

UC-006— Import Student Transcripts
Primary Actor: Registrar Office
Preconditions:

- User is authenticated as Registrar staff.
- Transcript data is available in supported format (CSV/Excel).

Main Flow:
1. Registrar staff selects "Import Transcripts."
2. System displays import interface.
3. Registrar staff selects transcript file.
4. System validates file format.
5. System reads transcript data.
6. System validates data fields:
   - Student IDs exist
   - Course codes are valid
   - Grades are valid
   - Credits are numeric
7. System displays preview of import.
8. Registrar staff reviews preview.
9. Registrar staff confirms import.
10. System imports transcript records.
11. System triggers audit recalculation for affected students.
12. System displays import summary (records imported, errors).

Alternative Flows:

- If file format is invalid → system rejects file with error message.
- If validation fails for some records → system displays error list, allows correction.
- If student ID not found → system flags record as error.
- If course code not found → system flags record as error but allows import with warning.
- If import is cancelled → system discards import without changes.

Postcondition:
Transcript data is imported and student audits are recalculated.

UC-005 — Identify At-Risk Students
Primary Actor: Faculty Advisor

Preconditions:
• Advisor is authenticated.
• Advisor has assigned advisees in the system. • Student audit data is available.

Main Flow:

1)Advisor selects "At-Risk Students" from advisor dashboard.
2)System retrieves audit data for all assigned advisees.
3)System applies at-risk criteria:
- CGPA below 6.0
- Missing required courses for current semester level
- Credit deficiency of more than 6 credits
- No courses completed in previous semester

4)System generates at-risk student list with:
- Student name and ID
- Risk indicators (low CGPA, credit deficiency, missing courses)
- Current CGPA and total credits earned • Deficiency details

5)System displays list sorted by risk severity (high to low).
6)Advisor reviews at-risk students.
7)Advisor can select individual student to view detailed audit report.

Alternative Flows:
• If no advisees are assigned → system displays message "No advisees assigned to you."
• If no at-risk students found → system displays message "All advisees are on track."
• If advisor selects a student → system navigates to student's detailed audit (UC-002 view).

Postcondition: Advisor has identified at-risk students and can take appropriate intervention actions.

UC-007 — Define Catalog Requirements
Primary Actor: Registrar Office

Preconditions:
• User is authenticated as Registrar staff.
• Program and catalog year information is available.

Main Flow:

1)Registrar staff selects "Define Catalog Requirements."
2)System displays catalog management interface.
3)Registrar staff selects program (BTech/MTech) and department (CCE/CSE/ECE/MME).
4)Registrar staff selects catalog year.
5)System displays requirement categories:
• Core courses
• Program Electives (PE)
• Departmental Electives (DE)
• Open Electives (OE)
• HSS courses
• Major requirements
• Minor requirements

6)For each category, Registrar staff enters:
• Minimum credit requirement
• List of applicable courses (if fixed)

• Course selection rules (if flexible)
• Minimum GPA requirement (if applicable)
7)Registrar staff defines prerequisite relationships between courses.
8)Registrar staff defines course equivalencies and substitutions.
9)System validates requirements (total credits ≥ 120 for BTech).
10)Registrar staff saves catalog requirements.
11)System confirms catalog saved successfully.
12)Alternative Flows:
• If total credits < 120 → system displays warning "Minimum 120 credits required for BTech."
• If duplicate catalog already exists → system asks "Catalog for this program/department/year already exists. Overwrite?"
• If validation fails → system displays error messages and prevents saving.
• If Registrar cancels → system discards changes without saving.

Postcondition: Catalog requirements are defined and saved. Students under this catalog will be audited against these requirements.

20. User Stories

Student User Stories

US-001
As a student, I want to log in securely so that I can access my degree audit information.

US-002
As a student, I want to view my degree progress dashboard so that I can see how close I am to graduation.

US-003
As a student, I want to see my completed courses so that I know what requirements I have already satisfied.

US-004
As a student, I want to see my remaining requirements so that I can plan my future semesters.

US-005
As a student, I want to perform what-if analysis for changing my major so that I can make informed decisions about my program.

US-006
As a student, I want to see if I am eligible for graduation so that I can prepare for degree conferral.

US-007
As a student, I want to download my audit report so that I can share it with my advisor.

US-008
As a student, I want to view my transfer credit evaluations so that I understand how my previous coursework applies.

Advisor User Stories

US-009
As a faculty advisor, I want to view progress for all my advisees so that I can monitor their academic performance.

US-010
As a faculty advisor, I want to identify at-risk students so that I can provide timely intervention.

US-011
As a faculty advisor, I want to see detailed audit reports for individual advisees so that I can provide informed guidance.

US-012
As a faculty advisor, I want to generate reports for all my advisees so that I can track overall advising effectiveness.

Registrar User Stories

US-013
As a registrar staff member, I want to import student transcripts so that audit calculations are based on current data.

US-014
As a registrar staff member, I want to define catalog requirements so that students are audited against correct criteria.

US-015
As a registrar staff member, I want to generate graduation eligibility reports so that I can process degree conferral.

HOD User Stories

US-016
As a HOD, I want to view department-level analytics so that I can assess program effectiveness.

US-017
As a HOD, I want to see cohort progress reports so that I can identify curriculum issues.

Admission Cell User Stories

US-018
As an admission cell staff member, I want to record transfer credits so that lateral entry students receive proper credit.

US-019
As an admission cell staff member, I want to map transfer courses to LNMIIT requirements so that audits reflect all earned credits.

Dean User Stories

US-020
As a dean, I want to approve exception requests so that special cases are handled appropriately.

System Administrator User Stories
US-021

As an administrator, I want to create, update, and manage user accounts so that users can access the system with the appropriate roles and permissions**.**

US-022
As an administrator, I want to configure system-wide settings so that the degree audit system can be maintained according to institutional requirements.

21. Acceptance Criteria

US-002 — View Degree Progress Dashboard

ID	Acceptance Criterion
AC-01	Student must be authenticated
AC-02	Dashboard must display overall progress percentage
AC-03	Dashboard must show status for each requirement category
AC-04	Completed courses must be listed with grades and credits
AC-05	In-progress courses must be clearly indicated
AC-06	Remaining requirements must be listed
AC-07	Graduation eligibility status must be displayed prominently
AC-08	Visual indicators (colors) must clearly distinguish satisfied/in-progress/not-started

US-013 — Import Student Transcripts

ID	Acceptance Criterion
AC-01	User must have Registrar role
AC-02	System must accept CSV or Excel format
AC-03	System must validate file format before import
AC-04	System must validate all data fields (student ID, course code, grade, credits)
AC-05	System must display preview before final import
AC-06	System must provide import summary showing successes and errors
AC-07	System must recalculate audits for affected students
AC-08	Import must be atomic (all or nothing if critical errors found)

22. Requirements Prioritization

Must Have

- User authentication and authorization
- Role-based access control
- Transcript import functionality
- Catalog requirement definition
- Automated audit calculation
- Student progress dashboard
- Requirement breakdown display
- Completed courses display
- Remaining requirements display
- Graduation eligibility determination
- Basic audit report generation
- Advisor view of advisee progress

Should Have

- Transfer credit recording and mapping

- Course substitution tracking
- At-risk student identification
- Multi-student advisor reports
- Department-level analytics
- Report download/export functionality
- Prerequisite verification
- Multi-catalog year support

Could Have

- Advanced visual analytics and charts
- Automated email notifications for progress milestones
- Course planning recommendations
- Prerequisite chain visualization
- Cohort comparison reports
- Historical trend analysis
- Bulk data import/export tools

Won't Have in Initial Release

- Mobile native application
- Integration with external university systems
- AI-based course recommendations
- Automated degree conferral workflow
- Real-time grade sync from faculty systems
- Student self-service transcript upload
- Blockchain-based credential verification

23. Requirements Traceability

Requirements Traceability Matrix (RTM) — Initial Version:

Requirement	User Story	Use Case	Test
FR-001	US-001	UC-001	TBD
FR-018	US-002	UC-002	TBD
FR-025	US-002	UC-002	TBD
FR-031	US-018	UC-009	TBD
FR-032	US-019	UC-017	TBD
FR-046	US-007	UC-003	TBD
FR-041	US-011	UC-018	TBD
FR-043	US-015	UC-008	TBD
FR-013	US-013	UC-006	TBD
FR-006	US-014	UC-007	TBD
FR-048	US-021	UC-013	TBD
FR-036	US-009	UC-004	TBD
FR-039	US-012	UC-016	TBD
FR-050	US-022	UC-019	TBD
FR-057	US-020	UC-012	TBD

Note: The TBD (To Be Determined) entries will be filled in during the testing phase when specific test cases are designed.

24. Requirements Validation

Requirements should be evaluated against the following criteria:

Correctness

Does it represent a genuine stakeholder need?

- All requirements derived from stakeholder interviews and project objectives
- Requirements align with LNMIIT's actual academic processes

Unambiguity

Can it have only one reasonable interpretation?

- Requirements use specific terminology (catalog, transcript, audit)
- Each requirement has clear expected behavior

Completeness

Does it contain sufficient information?

- Requirements specify what, not how
- Inputs, outputs, and conditions are defined
- Edge cases and exceptions are addressed

Consistency

Does it conflict with another requirement?

- No requirement contradicts another
- Business rules are consistent across functional requirements
- Data requirements align with functional needs

Verifiability

Can we test whether it has been satisfied?

- Each requirement can be verified through testing
- Acceptance criteria are defined for user stories
- Success criteria are measurable

Feasibility

Can it realistically be implemented?

- Technical constraints are considered
- Resource limitations are acknowledged
- Complexity is manageable within scope

Traceability

Can we identify its origin and follow it through development?

- Requirements traced to stakeholders and objectives
- Requirements mapped to use cases and user stories
- RTM maintains linkage to testing

Prioritization

Do we know how important it is?

- MoSCoW prioritization applied
- Must-have requirements identified for v1.0
- Optional features deferred appropriately

Modifiability

Can it be changed without unnecessarily affecting the entire SRS?

- Requirements are modular and independent where possible
- Changes to one requirement minimize cascade effects
- Traceability supports impact analysis

25. Example of Requirement Quality

Poor
The system should work fast and be easy to use for students.
Issues:

- Ambiguous ("fast" and "easy" are subjective)
- Not verifiable (no measurable criteria)
- Not specific (what functionality?)

Better

FR-025 — Student Dashboard
The system shall provide students with a visual progress dashboard.
and
NFR-002 — Audit Calculation Performance
Audit calculation should complete within 5 seconds for a typical student transcript.
Why Better:

- Specific functionality identified
- Measurable performance criterion
- Clear, testable requirement
- Uses "shall" for mandatory functionality

26. Requirement Change Management

Because the LNMIIT Degree Audit System uses Agile development, requirements can change.

Change Management Process
1. **Change Request Received**
   - Source: Stakeholder feedback, testing, new regulations
2. **Impact Analysis**
   - Assess affected requirements, use cases, design
   - Evaluate effort and risk
3. **Review and Approval**
   - Team discusses feasibility
   - Product owner prioritizes
   - Stakeholders approve if needed
4. **Requirement Update**
   - Update SRS document
   - Update affected sections
   - Maintain version history
5. **Backlog Update**
   - Add/modify user stories
   - Adjust sprint backlog
6. **Prioritization**
   - Re-prioritize using MoSCoW
   - Adjust release plan
7. **Sprint Planning**

- Include in upcoming sprint if high priority
- Or defer to future sprint

8. **Design Change**
   - Update design documents
   - Update diagrams if needed
9. **Implementation**
   - Develop according to updated requirements
   - Follow coding standards
10. **Testing**
    - Update test cases
    - Verify change implementation
    - Regression testing
11. **Release**
    - Deploy to production
    - Update user documentation
12. **Traceability Update**
    - Update RTM
    - Maintain audit trail

Example Change Request

Change Request: Add support for certificate programs in degree audit

Impact Analysis:

- Affected Requirements: FR-007, FR-020, FR-028
- New Requirements:Extend certificate requirements to include certificate-specific eligibility and completion criteria.
- Design Impact: Database schema needs certificate tables
- Testing Impact: New test cases for certificate audit
- Effort: 2-3 sprints

Decision: Approved for Release 1.1 (deferred from v1.0 to focus on core degree audit)

27. Future Requirements

Potential future requirements for subsequent releases:

- **Real-time grade sync** — Integrate directly with faculty grade entry system
- **Mobile application** — Native iOS and Android apps
- **Advanced course recommendations** — AI-based suggestions for optimal course sequences
- **Automated notifications** — Email/SMS alerts for progress milestones
- **Integration with academic advising system** — Shared advising notes and recommendations
- **Blockchain credentials** — Tamper-proof degree verification
- **Multi-institution support** — Support for cross-institution degree programs
- **Course planning wizard** — Interactive semester-by-semester planner
- **Peer comparison** — Anonymous comparison with cohort progress
- **Certificate programs** — Full support for certificate audit
- **Micro-credentials** — Support for badges and skill certifications
- **Advanced analytics** — Predictive analytics for graduation timeline
- **API for external systems** — RESTful API for third-party integration
- **Self-service transcript upload** — Allow students to submit external transcripts

- **Automated degree conferral** — Workflow automation for graduation approval
- **Multi-language support** — Interface in multiple languages

28. SRS Baseline and Versioning

Initial Baseline: SRS v1.0

Version Control Information:
This baseline establishes the foundational requirements for automated degree audit processing, student progress tracking with requirement validation, and role-based reporting capabilities across all LNMIIT departments.

Version History

Version	Author	Description
0.1	Team T030	Initial draft based on Assignment 1
1.0	Team T030	Baseline release - Official SRS v1.0

Prepared by:

- Team T030
- Hatwar Manas Vinod (24UCC175)
- Dhruv Gupta (24UCC176)
- Aradhya Raina (24UCC177)
- Kota Sai Vignesh (24UCC178)

Individual Contribution

- **Hatwar Manas Vinod:** Contributed to requirements engineering, business rules, RTM preparation, and final document review.
- **Dhruv Gupta:** Contributed to user-class definition, user-story development, and acceptance criteria preparation.
- **Aradhya Raina:** Contributed to functional requirements, data requirements, and degree audit lifecycle design.
- **Kota Sai Vignesh:** Contributed to functional and non-functional requirements and detailed use-case development.

AI USAGE POLICY – STUDENT DECLARATION

We declare that:

- We have disclosed the use of AI tools, if any, during preparation of this assignment.

- AI-generated content, if used, has been reviewed, verified, edited and adapted by the team.
- We understand the project sufficiently to explain and defend all submitted content during a viva/discussion.
- The final document reflects the team's own project understanding and decisions.

AI Tool(s) Used: Claude
Purpose of AI Use:

- To review the project-definition document for completeness, consistency, and alignment with the assignment requirements
- Help in Search for Resources to understand, plan and decide on the project vision, problem statement, objectives, scope, and stakeholder descriptions of the degree Audit System.
- To identify areas that may require clarification or improvement.
- To Check for Grammatical Mistakes

Team ID: T030
Team Member Names:

- Aradhya Raina
- Kota Sai Vignesh
- Dhruv Gupta
- Hatwar Manas Vinod
-

Team Declaration:
We confirm that the information provided above is accurate and that we accept responsibility for the submitted work. All requirements have been reviewed and verified by the team to ensure they align with LNMIIT's academic structure and our project objectives established in Assignment 1.

Team Representative Signature/Confirmation:
DHRUV GUPTA

Software Development Laboratory
Course Code: CS314

Lab Assignment 3: Solution

1. Objective

The objective of this assignment is to model the behaviour of the Student Activity Tracking System using scenarios, a complete UML Use Case Diagram, and Activity Diagrams. The models are based on the requirements and detailed use cases defined in Assignment 2.
The three detailed use cases selected from Assignment 2 are:

- **UC-002 -- View Degree Audit**
- **UC-005 – Identify At-Risk Students**
- **UC-006 – Import Student Transcripts**

These three use cases represent key academic progress tracking and degree audit workflows of the system.

Tasks

2. Task 1—- Scenario

2.1 Scenario for UC-002 (View Degree Audit)

- **Actor(s) involved:** Student
- **Starting condition:** The student is authenticated, has an assigned catalog year, and their transcript is available in the system.
- **Main sequence of activities:**
  1. The student selects “View my Audit” on the degree audit system dashboard
  2. The system Shows the student's transcript.
  3. The system retrieves the applicable catalog requirements.
  4. The system calculates the student's Degree attainment requirement completion.
  5. The system displays the student's overall progress, category-wise status, completed courses, in-progress courses, and remaining requirements.
  6. The system displays the student's graduation eligibility status.
  7. The student reviews the displayed degree audit.
- **Important alternative/exception flow:**
  1. If the student's transcript is unavailable —> the system displays an appropriate message.
  2. If the catalog requirements are unavailable —-> the system informs the student.
  3. If the audit calculation fails —> the system displays the last successfully calculated audit, if available.
- **Expected outcome:** The student successfully views their current degree audit and understands their academic progress and remaining requirements.

2.2 Scenario for UC–005 (Identify At Risk-Students)

- **Actor(s) involved:** Faculty Advisor

- **Starting condition:** The Faculty Advisor is authenticated, has assigned advisees, and audit data for those students is available.
- **Main sequence of activities:**
    1. The Faculty Advisor selects “At-Risk Students” button on the degree audit system dashboard
    2. The system retrieves the degree audit information of the advisor's assigned students.
    3. The system applies the defined at-risk criteria.
    4. The system identifies students meeting one or more risk criteria.
    5. The system generates a list containing student details, risk indicators, CGPA, credits, and academic deficiencies.
    6. The system sorts the identified students according to the severity of their risk.
    7. The Faculty Advisor reviews the list.
    8. The Faculty Advisor may select an individual student to view their detailed audit.
- **Important alternative/exception flow:**
    1. If the Faculty Advisor has no assigned Students —> the system displays an appropriate message.
    2. If no students meet the at-risk criteria —-> the system indicates that no at-risk students were found.
- **Expected outcome:** The Faculty Advisor successfully identifies students who may be at academic risk and can review their individual academic progress for further intervention.

2.3 Scenario for UC-006 (Import Student Transcripts)

- **Actor(s) involved:** Registrar
- **Starting condition:** The Registrar is authenticated and has a transcript file in a supported CSV or Excel format.
- **Main sequence of activities:**
    1. The Registrar selects “Import Student Transcripts” on the degree audit system dashboard
    2. The system displays the transcript import interface.
    3. The Registrar selects the transcript file.
    4. The system validates the file format.
    5. The system reads and validates the transcript data, including student IDs, course codes, grades, and credits.
    6. The system displays a preview of the data and any validation results.
    7. The Registrar reviews the preview and confirms the import.
    8. The system imports the valid transcript data.
    9. The system triggers recalculation of the affected degree audits.
    10. The system displays an import summary.
- **Important alternative/exception flow:**
    1. If the file format is invalid —-> the system rejects the file.
    2. If validation errors are found —-> the system displays the errors for correction.
    3. If a student or course cannot be identified —-> the system reports the corresponding issue.
    4. The Registrar may also cancel the import before confirmation.

- **Expected outcome:** The student transcript data is successfully imported into the system and the affected degree audits are recalculated.

Tasks

3. Task 2—– UML use case diagram

3.1 Actors:-

1. **Student:**
   The student logs into the system, views their degree audit, monitors completed and remaining requirements, views their academic progress, and downloads their audit report.
2. **Faculty Advisor :**
   The Faculty Advisor views advisee progress and audit information, identifies at-risk students, monitors academic progress, and generates advisor reports.
3. **Head of Department (HOD) :**
   The HOD accesses department-level analytics, monitors graduation eligibility and requirement compliance, and generates department reports.
4. **Registrar Office :**
   The Registrar manages catalog requirements, imports and updates student transcripts, generates graduation eligibility reports, and manages catalog-year requirements.
5. **Registrar Office :**
   The Admission Cell records transfer credits, maps transfer courses, manages course substitutions, and reviews their impact on degree requirements.
6. **Dean of Academic Affairs :**
   The Dean reviews academic policy reports, approves exception requests, and monitors academic standards and trends.
7. **System Administrator :**
   The System Administrator manages user accounts and roles, configures system settings, maintains system data, and monitors system activity and logs.

3.2 Use Cases

1. **UCC-001 – Login**
2. **UC-002 – View Degree Audit**
3. **UC-003 – Download Audit Report**
4. **UC-004 – View Advisee Progress**
5. **UC-005 – Identify At-Risk Students**
6. **UC-006 – Import Student Transcripts**
7. **UC-007 – Define Catalog Requirements**
8. **UC-008 – Generate Graduation Eligibility Report**
9. **UC-009 – Record Transfer Credits**
10. **UC-010 – Approve Course Substitution**
11. **UC-011 – View Department Analytics**
12. **UC-012 – Approve Exception Request**
13. **UC-013 – Manage User Accounts**
14. **UC-014 – View System Logs**
15. **UC-015 – Update Catalog Requirements**
16. **UC-016 – Generate Advisor Reports**

17. **UC-017 – Map Transfer Courses**
18. **UC-018 – View Advisee Audit Report**
19. **UC-019 – Configure System Settings**

The three detailed use cases from Assignment 2 are clearly identified in the diagram with a separate color(green)

- **UC-002 -- View Degree Audit**
- **UC-005 – Identify At-Risk Students**
- **UC-006 – Import Student Transcripts**

USE CASE DIAGRAM

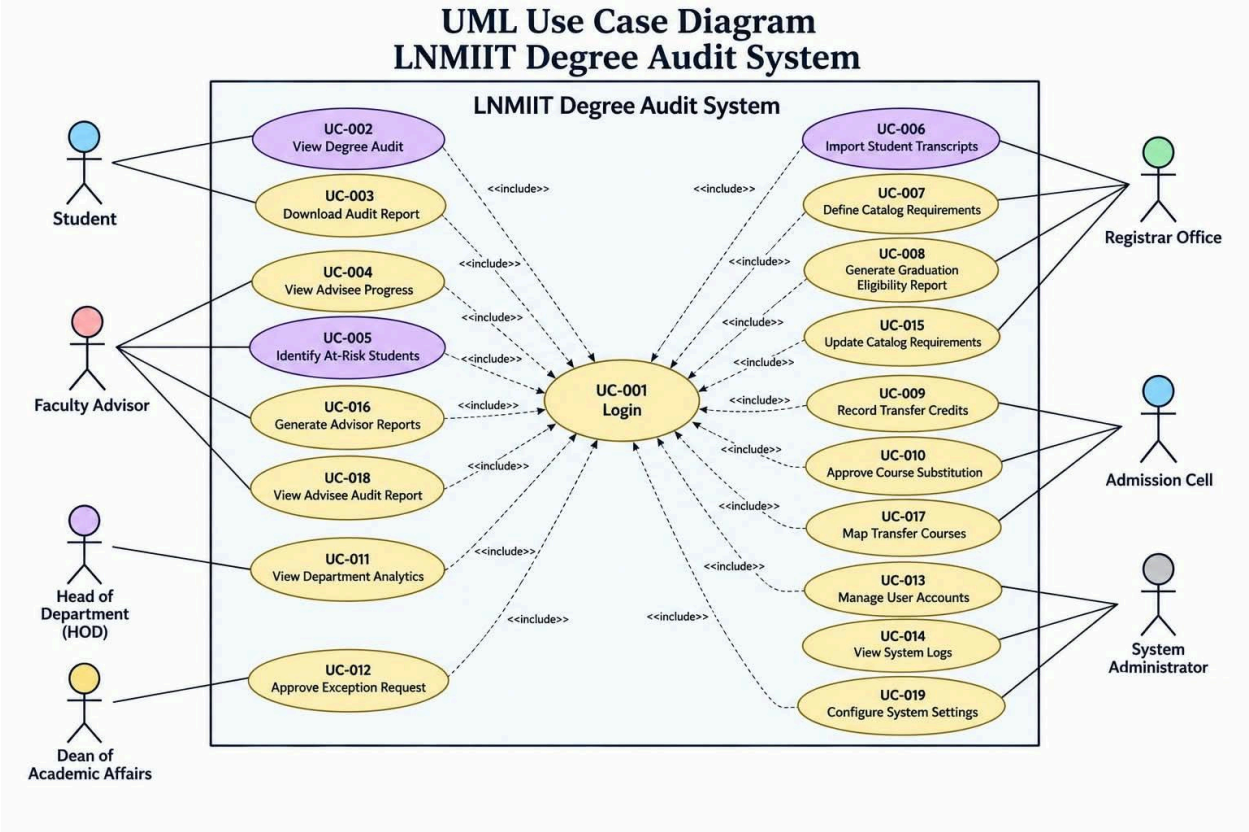


Figure1: use case diagram for Degree audit system

Tasks

4. Task 3—– Activity diagram

4.1 Activity Diagram: UC-002 -- View Degree Audit

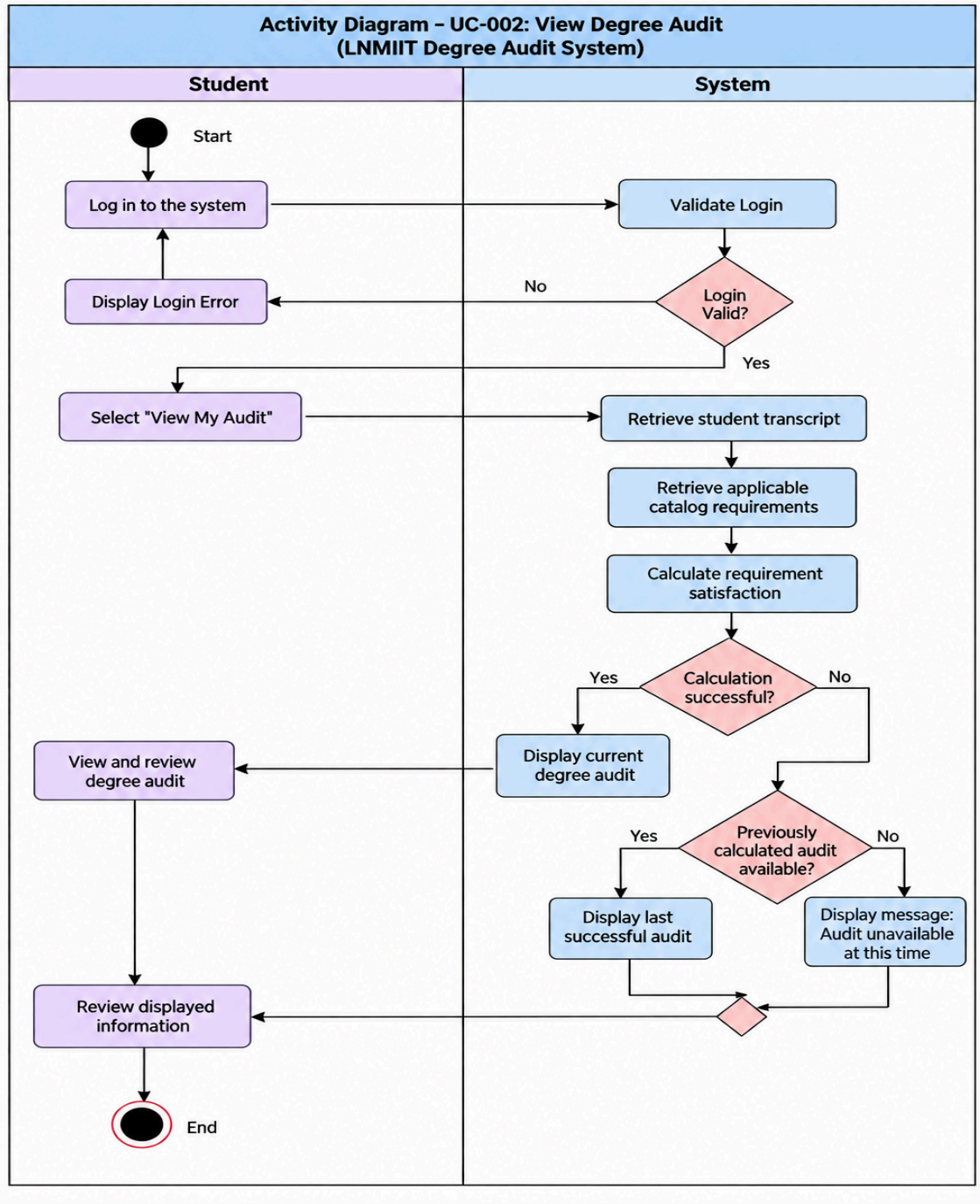


Figure2: Activity diagram for Use Case 002

4.1 Activity Diagram: UC-005 -- Identify At-Risk Students

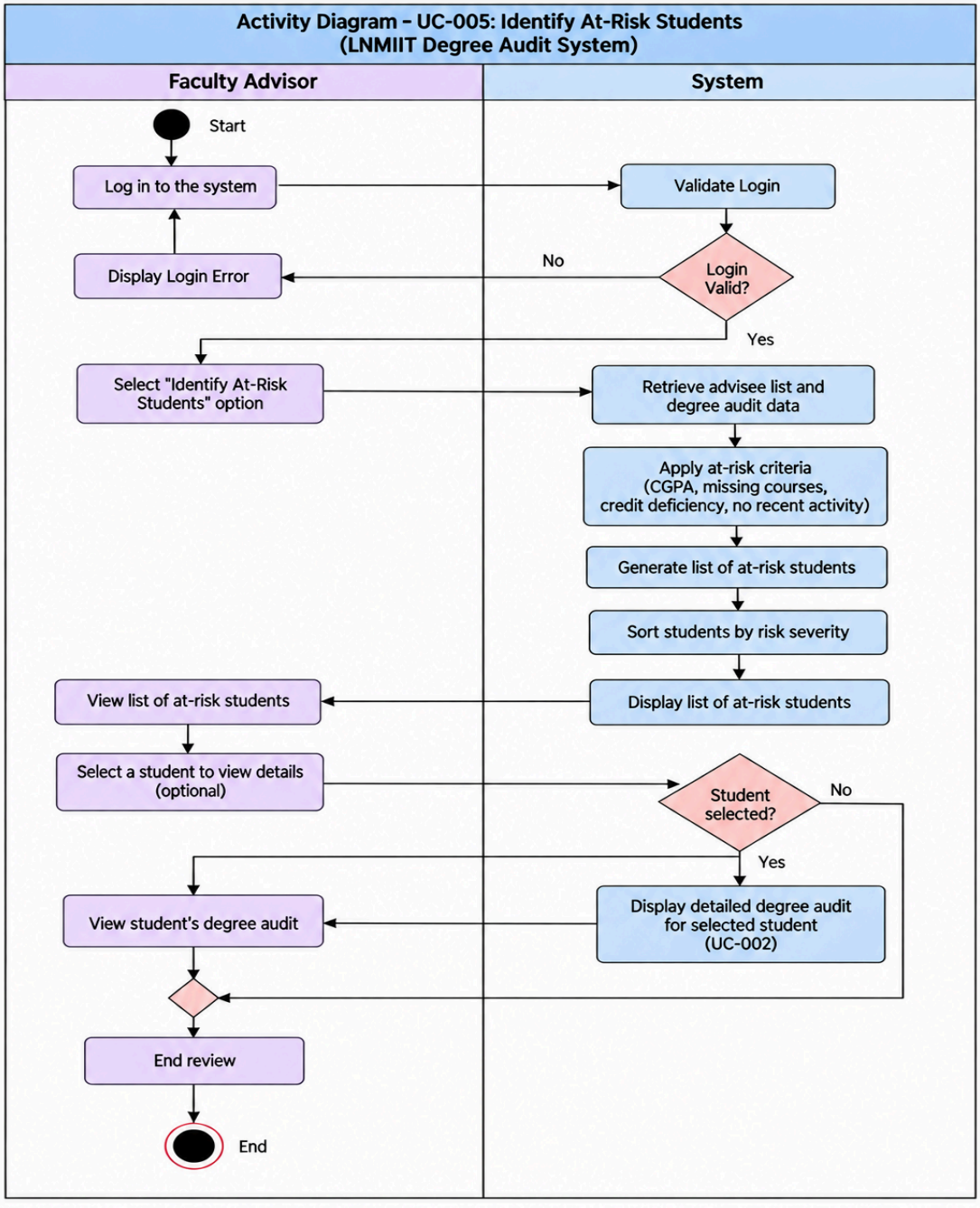


Figure3: Activity Diagram for Use case 005

4.1 Activity Diagram: UC-006 -- Import Student Transcripts

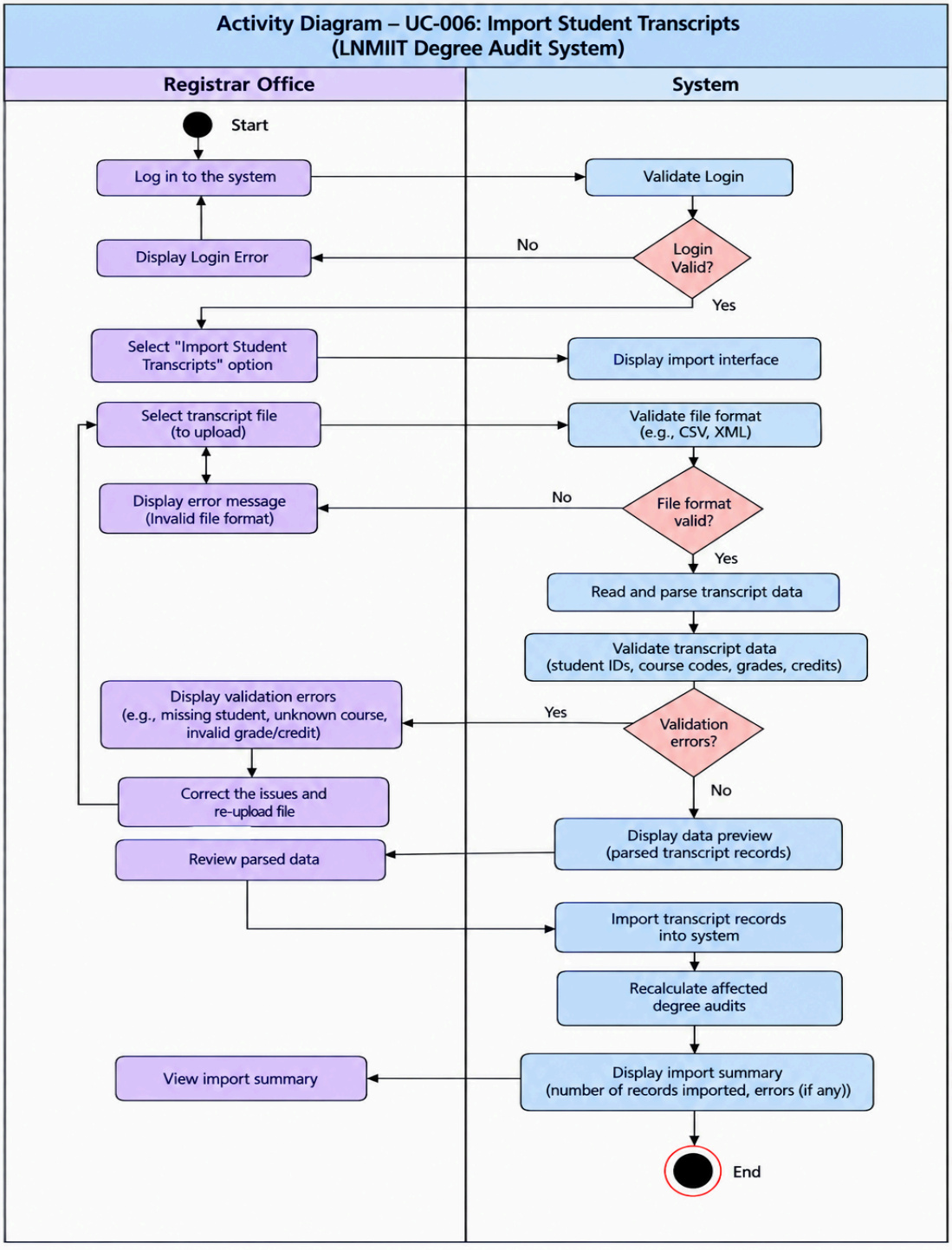


Figure4: Activity Diagram for Use Case 006

Prepared by:

- Team T030
- Hatwar Manas Vinod (24UCC175)
- Dhruv Gupta (24UCC176)
- Aradhya Raina (24UCC177)
- Kota Sai Vignesh (24UCC178)

Individual Contributions

- **Hatwar Manas Vinod:** Contributed to scenario preparation for the selected use cases, including main flows and alternative/exception scenarios, and assisted with final document review.
- **Dhruv Gupta**: Contributed to the UML Use Case Model, including actor identification, use-case associations, and organization of the complete Use Case Diagram.
- **Aradhya Raina:** Contributed to the Activity Diagram for UC-002 – View Degree Audit, including activity flow, decision branches, exception handling, and swimlane modelling.
- **Kota Sai Vignesh**: Contributed to the Activity Diagrams for UC-005 – Identify At-Risk Students and UC-006 – Import Student Transcripts, including workflow, decision/merge logic, and swimlane modelling.

AI USAGE POLICY – STUDENT DECLARATION

We declare that:

- We have disclosed the use of AI tools, if any, during preparation of this assignment.
- AI-generated content, if used, has been reviewed, verified, edited and adapted by the team.
- We understand the project sufficiently to explain and defend all submitted content during a viva/discussion.
- The final document reflects the team's own project understanding and decisions.

AI Tool(s) Used: Claude

Purpose of AI Use:

- To review the project-definition document for completeness, consistency, and alignment with the assignment requirements
- Help in Search for Resources to understand, plan and decide on the project vision, problem statement, objectives, scope, and stakeholder descriptions of the degree Audit System.
- To identify areas that may require clarification or improvement.
- To Check for Grammatical Mistakes

Team ID: T030

Team Member Names:

- Aradhya Raina
- Kota Sai Vignesh
- Dhruv Gupta

- Hatwar Manas Vinod

Team Declaration:
We confirm that the information provided above is accurate and that we accept responsibility for the submitted work. All requirements have been reviewed and verified by the team to ensure they align with LNMIIT's academic structure and our project objectives established in Assignment 1,2.

Team Representative Signature/Confirmation:
DHRUV GUPTA

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314

Lab Assignment 4: Solution

1. Objective

The objective of this assignment is to develop the static structural model of the LNMIIT Degree Audit System using UML Class and Object Diagrams.

The assignment focuses on identifying the important classes involved in the system, defining their attributes and operations, identifying relationships between classes, and creating an Object Diagram representing a particular system state.

The class and object models are designed consistently with the requirements, user stories, use cases, and activity diagrams developed in the previous assignments.

2. Task 1 – Class Identification

The following classes were identified for the Student Activity Tracking System based on the major functionalities and the three detailed use cases:

- UC-002 – View Degree Audit
- UC-005 – Identify At-Risk Students
- UC-006 – Import Student Transcripts

A total of 12 classes were decided

2.1 Classes

S. No.	Class Name	Important Attributes	Important Operations / Responsibilities	Traceability
1	Student	studentId, name, program, department, batchYear, catalogYear, currentSemester, status	viewDegreeAudit(), viewProgress()	UC-002, UC-005

2	Transcript	transcriptId, studentId, semester, grade, creditsEarned, status, importDate	validateData(), addCourseRecord(), getCourseHistory()	UC-002, UC-006
3	Course	courseCode,courseName ,department, credits, courseType,prerequisites	validateCourseCode(), checkPrerequisite()	UC-002, UC-006
4	Catalog	catalogId, catalogYear, program, department, effectiveDate	getApplicableRequirements()	UC-002
5	Requirement	requirementId, category, description, minimumCredits, minimumGPA, courseList, rules	checkSatisfaction(), identifyDeficiency()	UC-002,UC-005
6	AuditReport	auditId, generatedDate, catalogYear, totalCreditsEarned, totalCreditsRequired, CGPA, graduationEligible, status	calculateProgress(), calculateGPA(), determineEligibility(), generateReport()	UC-002, UC-005
7	FacultyAdvisor	advisorId, name, department	viewAdvisees(), viewStudentAudit(), identifyAtRiskStudents()	UC-005
8	AdvisorAssignment	assignmentId, advisorId, studentId, assignmentDate, status	assignStudent(), getAssignedStudents()	UC-005
9	AtRiskAssessment	assessmentId, riskIndicators, riskLevel, deficiencyDetails	evaluateRisk(), identifyRiskFactors(), generateAtRiskList()	UC-005

10	TranscriptImport	importId, fileName, fileFormat, importDate, recordsImported, errorCount, status	validateFile(), validateRecords(), previewImport(), confirmImport(), generateSummary()	UC-006
11	RegistrarStaff	registrarId, name, department	importTranscript(), reviewImport(), confirmImport()	UC-006
12	UserAccount	userId, email, role, accountStatus, lastLogin	authenticate(), checkAccess()	UC-002, UC-005, UC-006

3. Task 2 – Class Diagram

3.1 Class Diagram

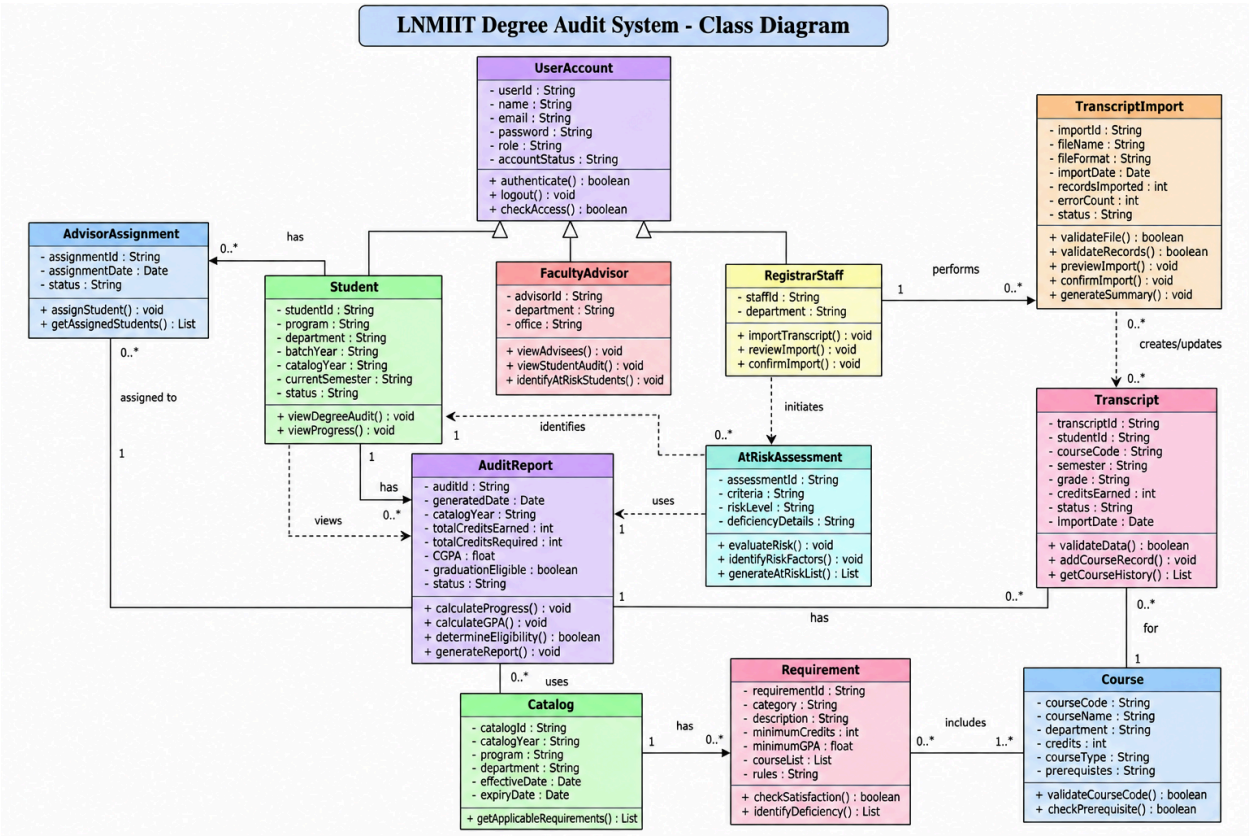


Figure 1: UML Class Diagram of the LNMIIT Degree Audit System

The central **UserAccount** class is generalized into **Student, FacultyAdvisor** and

RegistrarStaff, representing the different types of users interacting with the system. The **Student** class is associated with Transcript, **AuditReport** and **AdvisorAssignment** while **Catalog, Requirement ,** and **Course** represent the academic structures used for degree-audit processing. **AtRiskAssesment** supports the identification of students requiring academic attention, and **TranscriptImport** represents the transcript-import workflow performed by Registrar staff.

4. Task 3 – Object Diagram

4.1 Object Diagram

The Object Diagram represents a snapshot of the Student Activity Tracking System during UC-006 – Import Student Transcripts

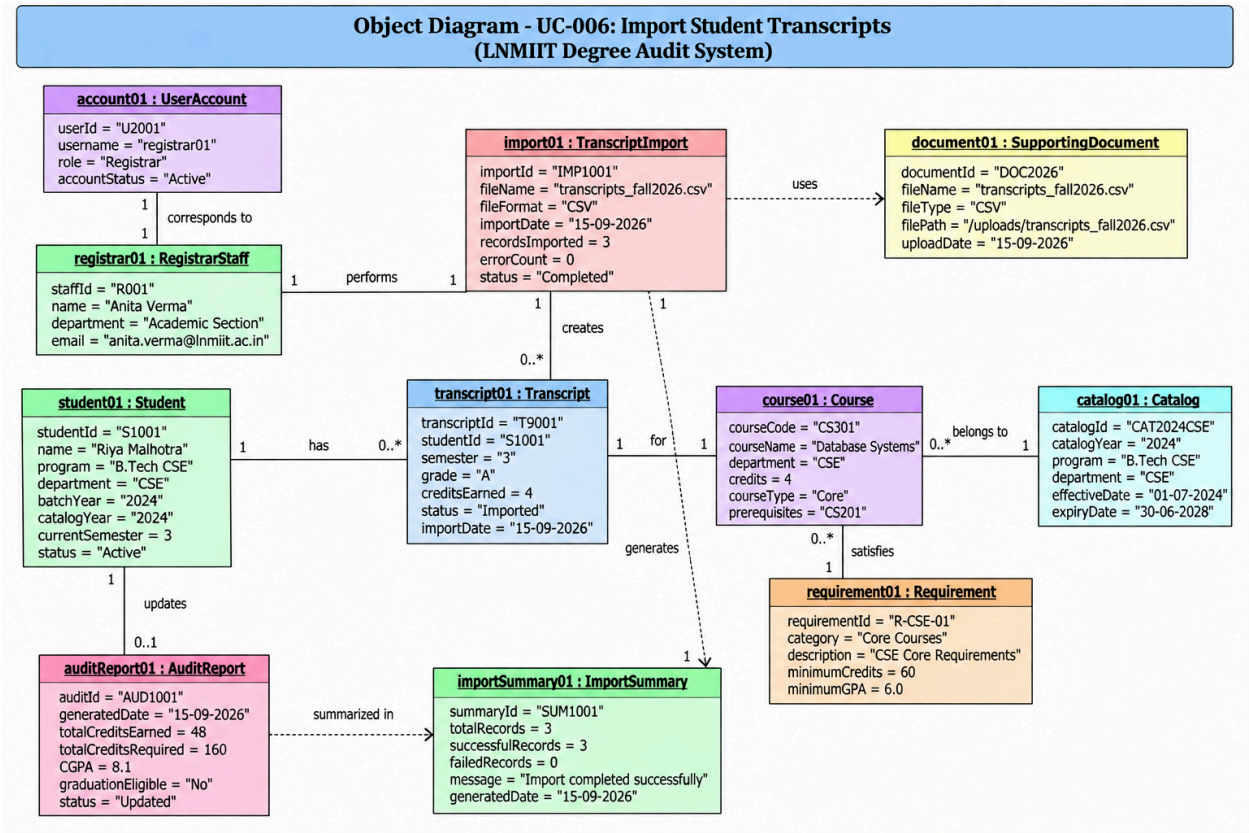


Figure 2: UML Object Diagram for UC-006-Import Student Transcript

4.2 Figure Description

The scenario represents a Registrar Staff member importing a student's transcript into the LNMIIT Degree Audit System using a supported CSV file. The system validates the file and transcript records, including the student ID, course codes, grades, and credits, before importing the valid records. After the import is confirmed, the student's transcript is updated and the corresponding audit report is recalculated. Finally, the system generates an import summary showing the number of records processed and any validation errors.

5. Task 4 – Relationship Identification and Justification

5.1 Relationship Summary

NO.	Classes(1st-2nd)	Relationship	Multiplicity	Justification
1	UserAccount – Student	Association	1 : 1	Each student has one user account used to access the system.
2	UserAccount – FacultyAdvisor	Association	1 : 1	Each faculty advisor is associated with a user account for system access.
3	UserAccount – RegistrarStaff	Association	1 : 1	Each Registrar staff member accesses the system through a user account.
4	Student – Transcript	Association	1 : 0..*	A student may have multiple transcript records covering different semesters.
5	Student – AuditReport	Association	1 : 0..*	A student can have multiple calculated audit reports over time.
6	Student – AdvisorAssignment	Association	1 : 0..*	A student can have advisor assignment records over their academic period.
7	FacultyAdvisor – AdvisorAssignment	Association	1 : 0..*	A faculty advisor can be assigned to multiple students.

8	AdvisorAssignment – Student	Association	1 : 1	Each advisor assignment refers to one specific student.
9	Transcript-Course	Association	1 : 1..*	A transcript contains records for courses completed or attempted by the student.
10	Catalog – Requirement	Composition	1 : 1..*	A catalog defines the academic requirements applicable to that catalog year.
11	Requirement – Course	Association	1 : 0..*	A requirement can specify multiple courses that satisfy it.
12	AuditReport – Catalog	Association	1 : 1	An audit report is calculated according to the student's applicable catalog.
13	AuditReport – Requirement	Association	1 : 1..*	The audit evaluates the student's progress against applicable requirements.
14	AtRiskAssessment – AuditReport	Association	1 : 1	At-risk assessment uses the student's calculated audit information.
15	FacultyAdvisor – AtRiskAssessment	Association	1 : 0..*	An advisor can review multiple at-risk assessments for assigned students.
16	RegistrarStaff – TranscriptImport	Association	1 : 0..*	Registrar staff member can perform multiple transcript imports.
17	TranscriptImport – Transcript	Composition	1 : 0..*	A transcript import creates/imports the transcript records associated with that import operation.
18	TranscriptImport –	Dependency	—---	The import process

	Student			depends on identifying the student to whom the transcript belongs.
19	TranscriptImport – Course	Dependency	—-----	The import process uses course information to validate imported course codes.
20	TranscriptImport – AuditReport	Dependency	—----	After a successful transcript import, the student's audit is recalculated.

6. Consistency With Previous Assignments

The Class and Object Models are consistent with the requirements, use cases, and activity diagrams developed in the previous assignments.

The overall workflow represented by the selected use cases includes:

Student Transcript → Transcript Validation and Import → Degree Audit Calculation → At-Risk Assessment

The Class Diagram represents the main entities and responsibilities involved in these processes. The Student class is associated with academic information stored in Transcript, while Course, Catalog, and Requirement support the evaluation of degree requirements. AuditReport represents the calculated degree progress and eligibility information used in UC-002 – View Degree Audit and UC-005 – Identify At-Risk Students. AtRiskAssessment supports the identification of students requiring academic attention, while TranscriptImport and RegistrarStaff represent the transcript import process described in UC-006 – Import Student Transcripts.

The Object Diagram provides a specific snapshot of the UC-006 – Import Student Transcripts process, showing a Registrar staff member importing a student's transcript, validating the transcript information, and updating the student's academic records for subsequent degree-audit processing.

7. Individual Contribution

- **HATWAR MANAS VINOD**: Class identification, Class Diagram design, and UML relationship identification.
- **DHRUV GUPTA**: Object Diagram design, class attributes and operations, and documentation.
- **ARADHYA RAINA**: UML diagrams creation and formatting using draw.io, and documentation review.
- **KOTA SAI VIGNESH:** Relationship and multiplicity verification, and consistency checking.

8.AI Usage Declaration

AI tools may be used as supporting tools for this assignment. The team is responsible for the correctness, consistency and understanding of all submitted classes, objects and relationships.

- We have disclosed the use of AI tools, if any.
- AI-generated suggestions have been reviewed, verified and adapted by the team.
- We understand and can explain the submitted classes, object diagram and relationships.
- We have verified that the UML models are consistent with our project's requirements, use
- cases and scope.
- The final submission represents our team's understanding and modeling decisions.

AI Tool(s) Used: Claude

Purpose of AI Use: claude was used for understanding UML concepts, organizing class attributes and operations, checking relationship types and multiplicities, and improving the clarity of the documentation.

The final class model, object model, diagrams, relationships, and project-specific decisions were reviewed and finalized by the team. The team takes responsibility for the correctness and content of the submitted work.

Team ID: T030

Team Member Names:

- Aradhya Raina(24UCC177)
- Kota Sai Vignesh(24UCC178)
- Dhruv Gupta(24UCC176)
- Hatwar Manas Vinod(24UCC175)

Team Declaration:

We confirm that the information provided above is accurate and that we accept responsibility for the submitted work. All requirements have been reviewed and verified by the team to ensure they align with LNMIIT's academic structure and our project objectives established in Assignment 1, 2 and 3.

Team Representative Signature/Confirmation: DHRUV GUPTA

Date: 20/09/2026

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314
Lab Assignment 5: Solution

1. Objective

This assignment extends the LNMIIT Degree Audit System models from Assignment 4 with object interactions, entity state changes, and major software components. The same three detailed use cases from Assignment 2 are used throughout, and the collaboration objects are instances of classes and relationships already identified in Assignment 4.

2. Task 1 – Collaboration Diagrams

The diagrams show the participating actor and objects, established object links, numbered messages, and the responsibility handled by each object. Message names use operations already listed for the corresponding Assignment 4 classes; attribute lookup is labeled where the class model defines no separate lookup operation.

2.1 Collaboration Diagram: UC-002 – View Degree Audit

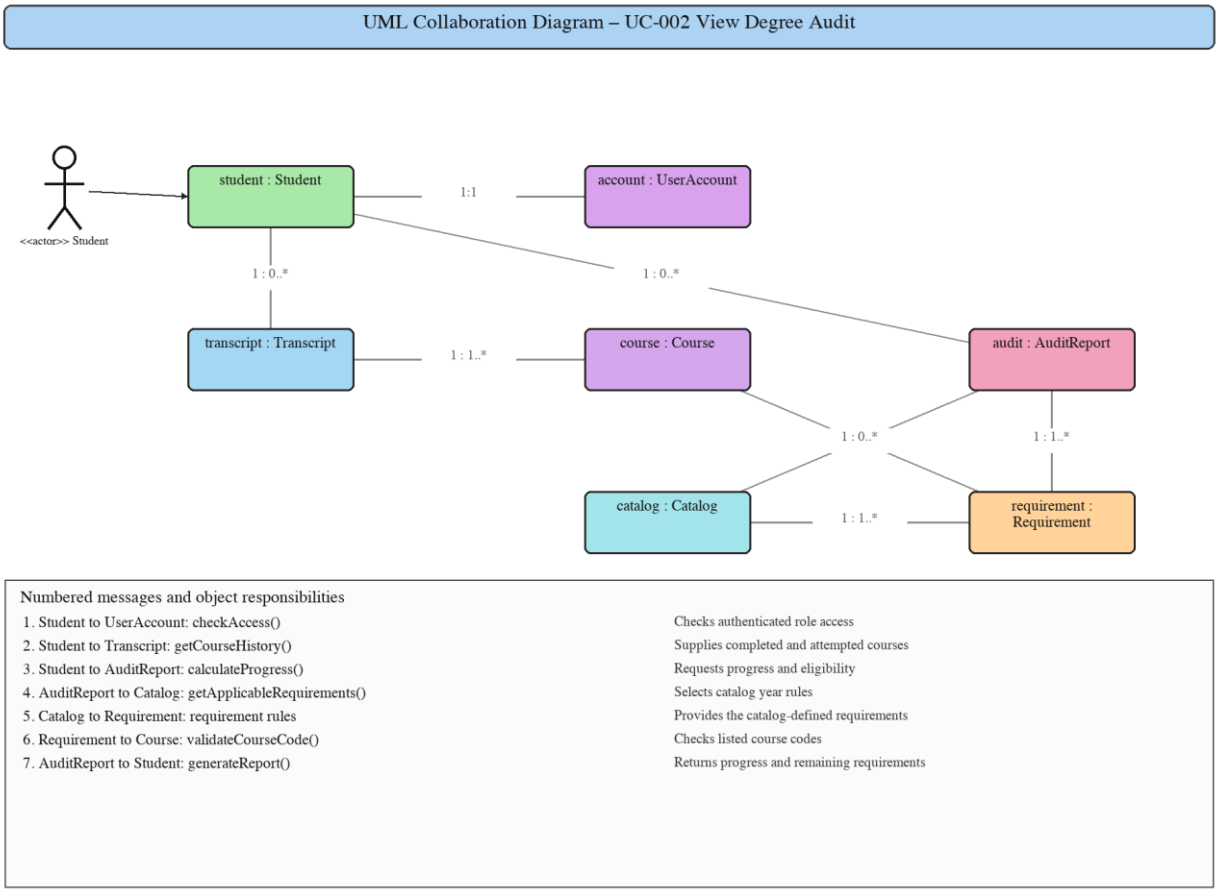


Figure 1: UML Collaboration Diagram for UC-002 – View Degree Audit

2.2 Collaboration Diagram: UC-005 – Identify At-Risk Students

The advisor retrieves assigned students, uses their audit information to evaluate the specified risk indicators, and reviews the resulting list. Selecting a student continues through the UC-002 detailed audit workflow.

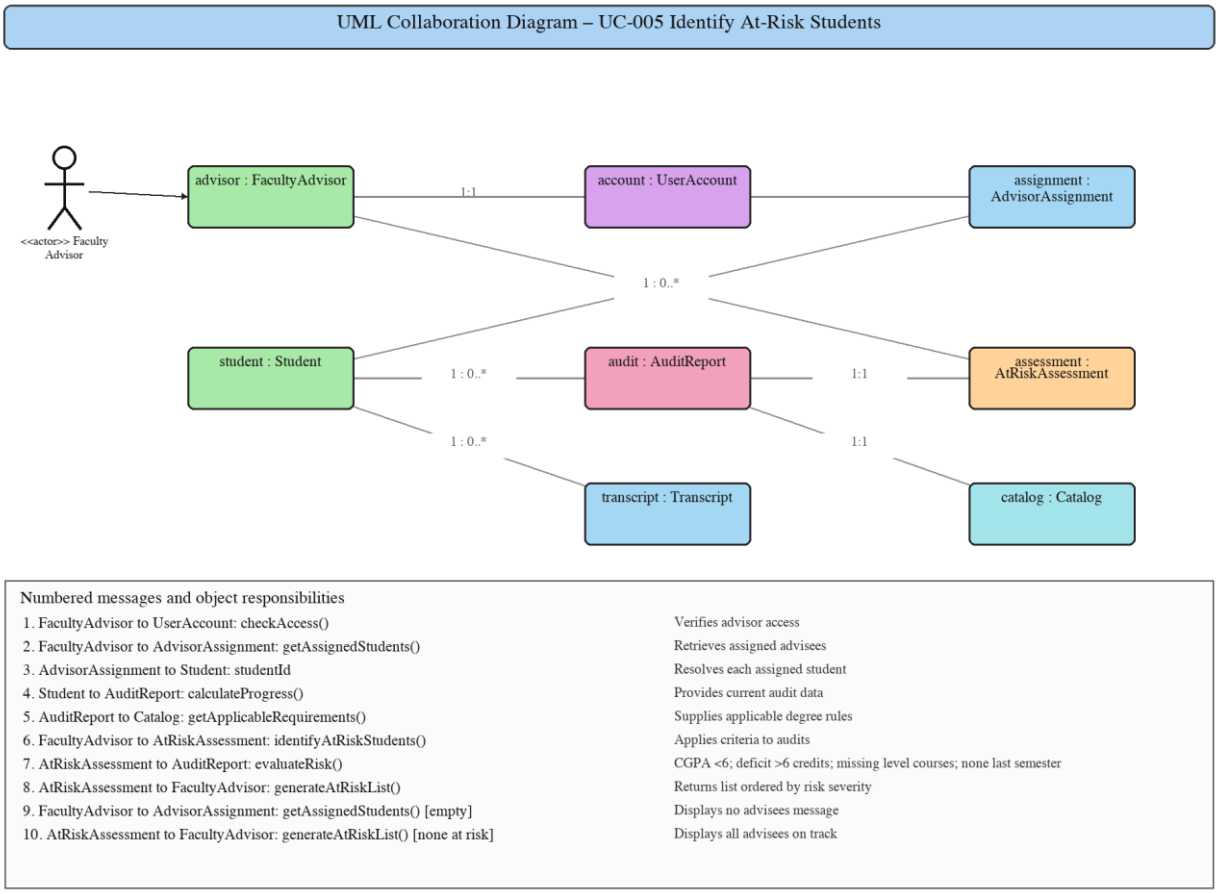


Figure 2: UML Collaboration Diagram for UC-005 – Identify At-Risk Students

2.3 Collaboration Diagram: UC-006 – Import Student Transcripts

The Registrar validates the file and record preview before confirming the import. Invalid data can be corrected or the import cancelled before confirmation; a successful import updates transcript records and triggers audit recalculation.

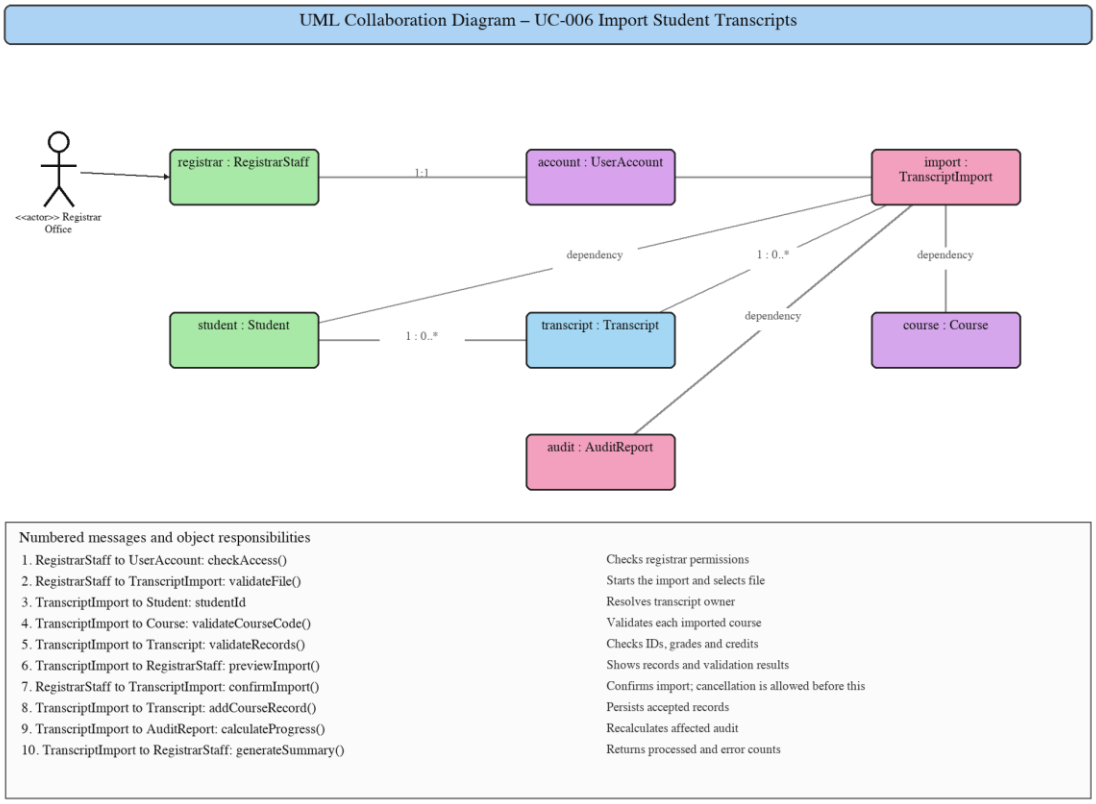


Figure 3: UML Collaboration Diagram for UC-006 – Import Student Transcripts

3. Task 2 – State Diagrams

The state models describe the lifecycle of two Assignment 4 entities. Guard conditions preserve the alternatives in the detailed use cases, including validation failure, cancellation, and cached audit display.

3.1 State Diagram: TranscriptImport

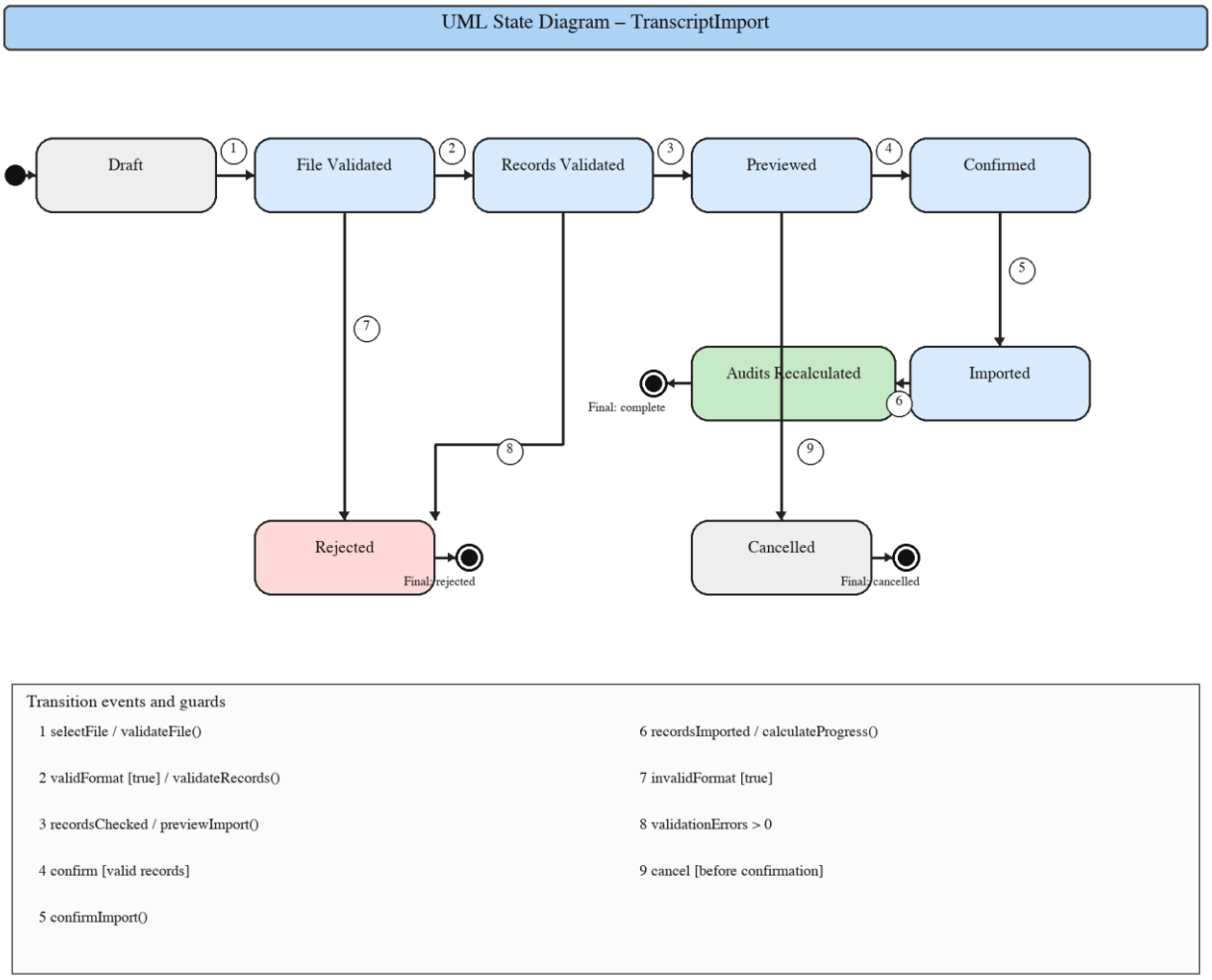


Figure 4: UML State Diagram for TranscriptImport

A transcript import moves from file selection through validation and preview. Confirmation starts persistence; successful records lead to recalculation of affected audits. Invalid format, validation errors, and cancellation terminate in separate final outcomes.

3.2 State Diagram: AuditReport

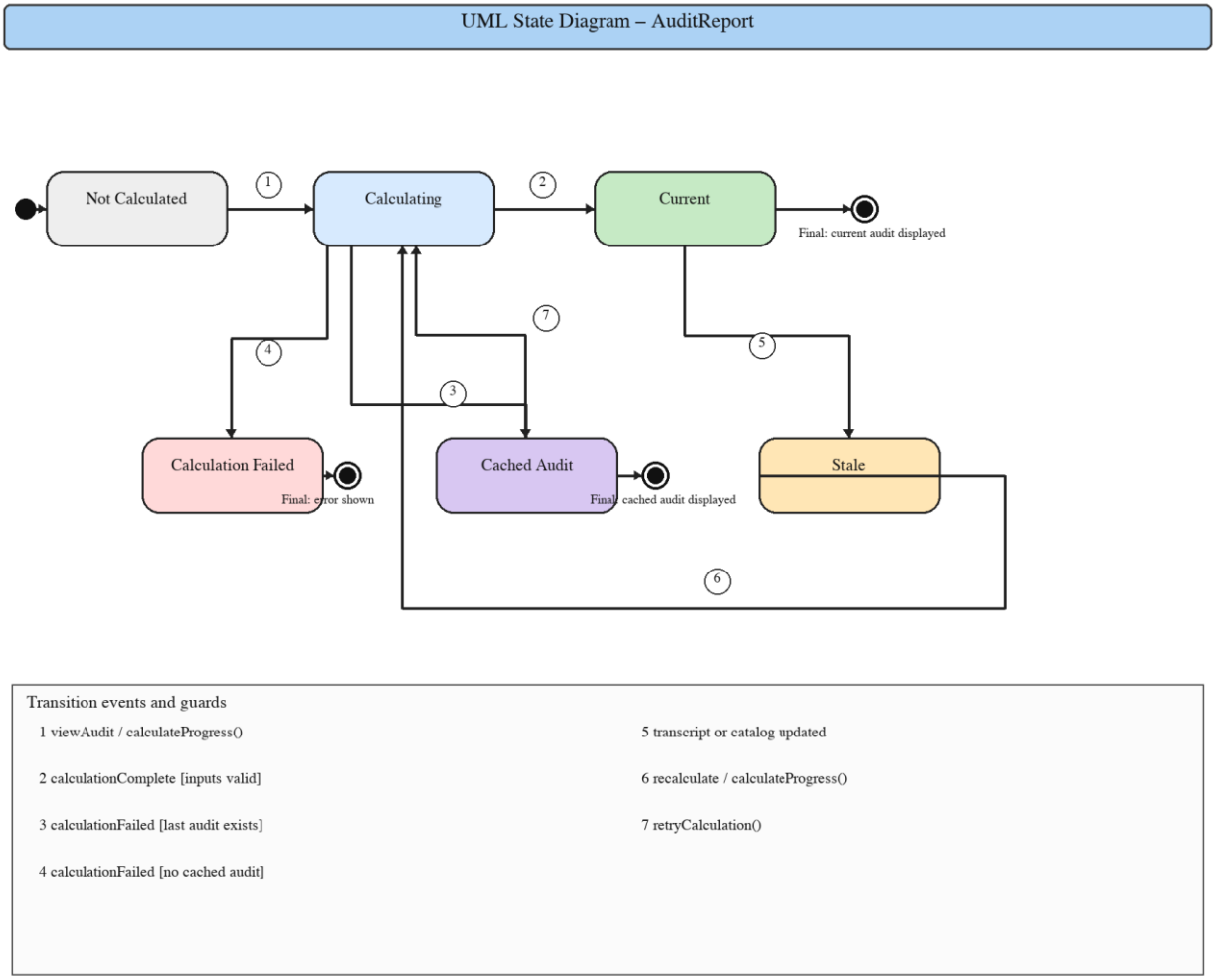


Figure 5: UML State Diagram for AuditReport

A requested audit is recalculated from the available transcript and catalog. A successful calculation produces a current report. If calculation fails, the system displays the last successful report when one exists; otherwise it reports that calculation failed. Transcript or catalog changes make a current report stale until recalculation.

4. Task 3 – Component Diagram

The components below support the three selected use cases. They group the Assignment 4 classes by responsibility, expose named interfaces, and show dependencies on academic data and the Registrar supplied CSV or Excel transcript file.

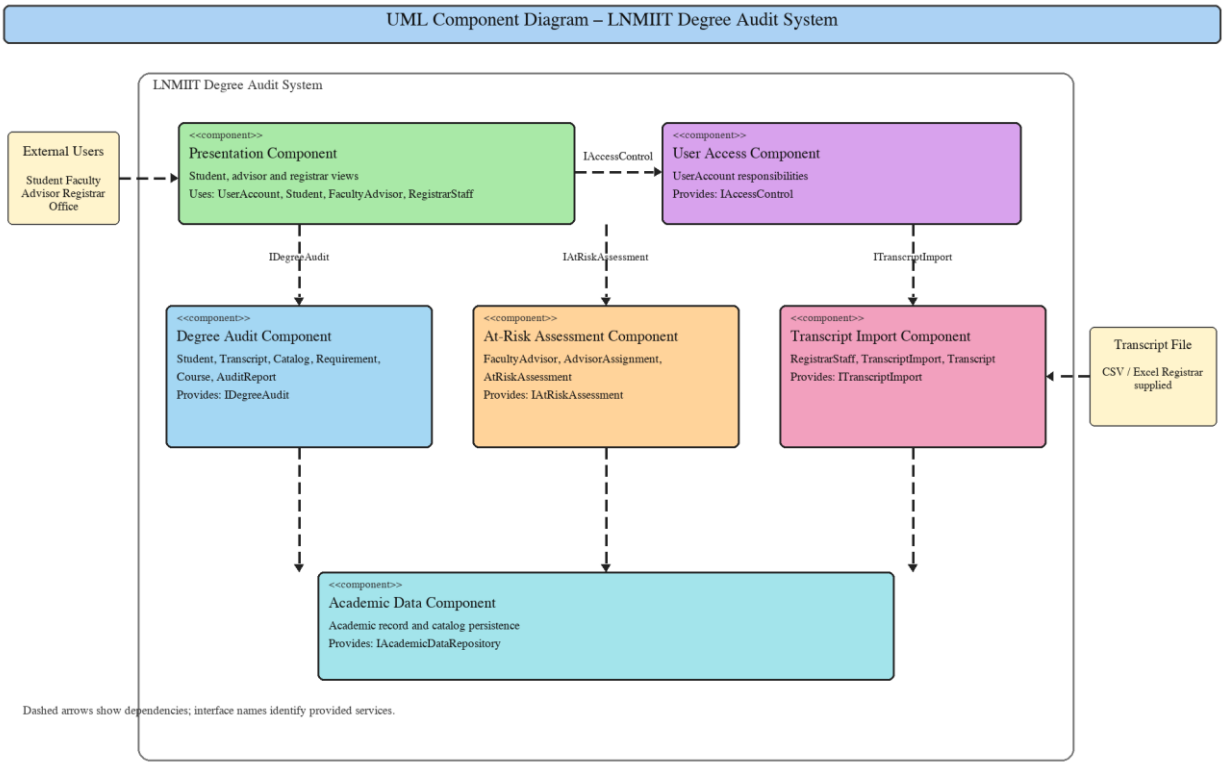


Figure 6: UML Component Diagram of the LNMIIT Degree Audit System

The Presentation Component supports student, advisor, and registrar interactions. The Degree Audit, At-Risk Assessment, and Transcript Import Components provide the use case services and access shared academic data through IAcademicDataRepository. UserAccount responsibilities are grouped in the access component; no external authentication service is assumed.

5. Consistency With Previous Assignments

The models continue the traceability chain established in the previous assignments: requirements and the same three detailed use cases lead to scenarios and activities, then to the Assignment 4 classes and relationships, collaboration objects, entity states, and software components.

UC-002 uses Student, UserAccount, Transcript, Course, Catalog, Requirement, and AuditReport. UC-005 uses FacultyAdvisor, UserAccount, AdvisorAssignment, Student, AuditReport, Transcript, Catalog, and AtRiskAssessment. UC-006 uses RegistrarStaff, UserAccount, TranscriptImport, Student, Transcript, Course, and AuditReport. Each participant connection follows an Assignment 4 association, composition, or dependency.

TranscriptImport uses the existing validateFile(), validateRecords(), previewImport(), confirmImport(), and generateSummary() responsibilities. AuditReport and AtRiskAssessment retain their existing calculation and risk-evaluation responsibilities. The component diagram groups these same responsibilities into implementation modules without adding domain classes.

6. Individual Contribution

The draft assigns diagram and consistency responsibilities across the four members. Confirm that this allocation matches each member's actual contribution before submission.

Team Member	Contribution
HATWAR MANAS VINOD (24UCC175)	UC-002 collaboration model and AuditReport state model; class traceability.
DHRUV GUPTA (24UCC176)	UC-006 collaboration model and TranscriptImport state model; transcript workflow.
ARADHYA RAINA (24UCC177)	UC-005 collaboration model; diagram preparation and document formatting.
KOTA SAI VIGNESH (24UCC178)	Component model; relationship consistency and final review.

7. AI Usage Declaration

OpenAI ChatGPT was used as a supporting tool to draft and organize the UML descriptions and diagrams. The team is responsible for reviewing and approving the models, confirming that members understand the submitted work, and finalizing this declaration before submission.

- AI assistance is disclosed; the tool and purpose are identified below.
- The submitted models must be reviewed and verified by the team against the project requirements, use cases, classes, objects, and scope.
- The team must understand and be able to explain the collaboration, state, and component diagrams.
- The final submission represents the team's modeling decisions after its review.

AI Tool(s) Used: OpenAI ChatGPT

Purpose of AI Use: Drafting the collaboration, state, and component models; organizing the assignment section. The final diagrams, relationships, and project-specific decisions were reviewed and finalized by the team. The team takes responsibility for the correctness and content of the submitted work.

Team ID: T030

Team Member Names:

- Aradhya Raina (24UCC177)
- Kota Sai Vignesh (24UCC178)
- Dhruv Gupta (24UCC176)
- Hatwar Manas Vinod (24UCC175)

Team Representative: Dhruv Gupta

Date: 29/09/2026